
Motivation

- Problem

- Autoregressive LLM reasoning에서 추가 compute는 accuracy를 올릴 수 있지만, 모든 문제에 동일하게 쓰면 비용이 크게 증가
- Compute-Accuracy trade-off를 조절하는 것이 핵심

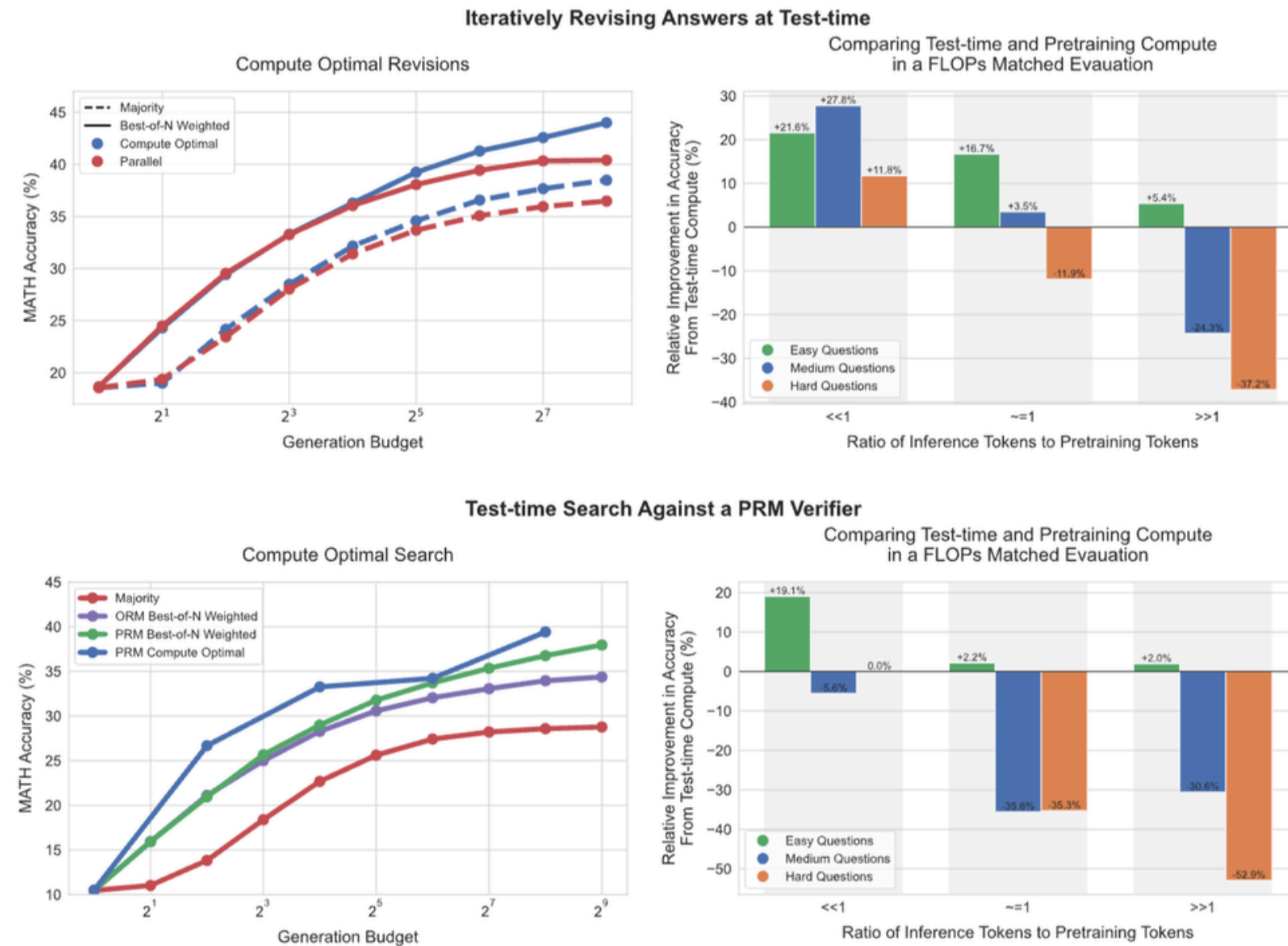
- Question:

- initial reasoning 상태를 보고 한 번의 routing 결정으로 computing을 adaptive하게 조절할 수 있는가?

- Idea:

- initial 답 생성 이후 STOP / VERIFY / SC-3 중 하나를 선택하는 one-step routing을 진행
-

Previous Work



- Test-time compute는 reasoning 성능을 올릴 수 있지만, 모든 문제에 많이 쓰는 것이 항상 최적은 아님
- 문제 난이도와 compute strategy에 따라 추가 compute의 효율이 달라짐을 확인

Background

- Previous Method:
 - CoT, Self-consistency, Verification , Majority Voting
 - 기존 방법론은 모든 query 혹은 task에 동일한 추가 compute를 적용하거나 여러 generation을 고정으로 실행
 - Our Method:
 - 현재 Initial answer를 보고 routing을 결정
 - STOP: initial answer 유지
 - VERIFY: 문제를 다시 풀고 기존 풀이/답을 검토하여 수정
 - SC-3: independent reasoning 2개 추가 생성 후 plurality vote
-

System Architecture

- Input: GSM8K problem & answer
 - Pipeline:
 - 1. initial prompt로 initial reasoning 생성
 - 2. initial answer extraction 및 correctness 판단
 - 3. VERIFY prompt 또는 SC-3 prompt로 추가 generation 수행
 - 4. STOP/VERIFY/SC-3 action별 final answer 산출
 - 5. Oracle utility 계산 및 controller 학습/평가
-

Experimental Setup

- Dataset:
 - GSM8k test split
 - Models:
 - Qwen2.5-1.5B-Instruct / Qwen2.5-3B-Instruct
 - Metric:
 - accuracy
 - avg total tokens
 - $\text{utility} = \text{correct} - \lambda * \text{cost}$
 - $\lambda = 0.1$
 - Baseline
 - Always STOP, Always VERIFY, Always SC-3
 - Random Router, Length Threshold Router
 - Oracle Router
 - Learned controllers: logistic, random forest, value predictor
-

Main results

방법	Accuracy	평균 토큰 수	Utility
Always STOP	0.253	299.68	0.222
Always VERIFY	0.370	916.16	0.275
Always SC-3	0.287	966.53	0.187
Oracle Router	0.473	435.65	0.428
RF Controller	0.430	484.83	0.380

- Oracle Router는 Always VERIFY보다 훨씬 적은 토큰을 사용하면서도 더 높은 accuracy와 utility를 달성
 - 문제별로 추가 compute의 가치가 다르다는 것을 확인
 - Initial reasoning state에는 추가 compute를 사용할 가치가 있는지에 대한 정보가 포함되어 있음
-

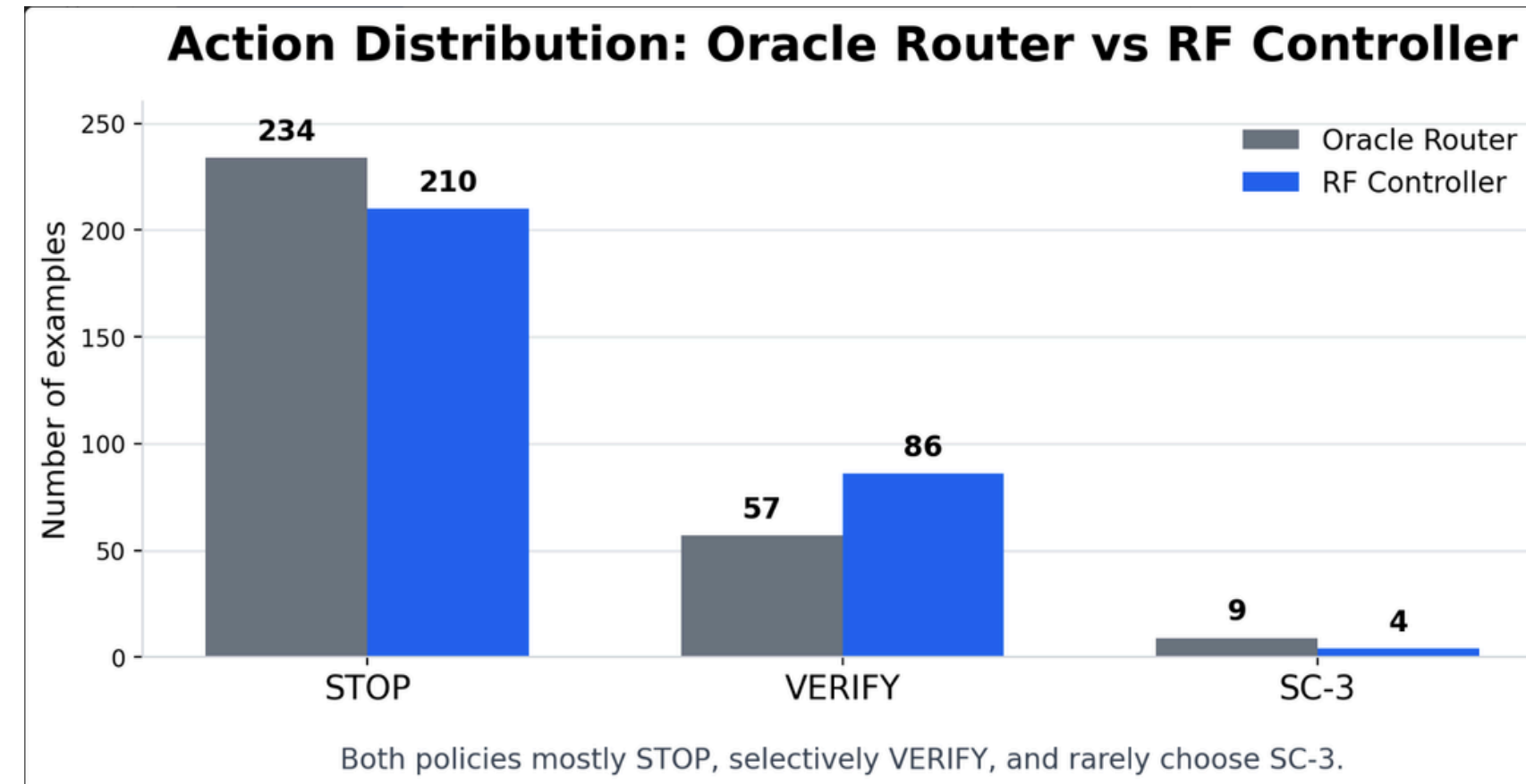
Controller Training

각 문제에서 STOP / VERIFY / SC-3를 모두 rollout한 뒤, utility가 가장 높은 action을 Oracle label로 사용

단계	내용
1. Rollout 생성	각 문제에 대해 STOP, VERIFY, SC-3 결과를 모두 수집
2. Oracle label 생성	<code>utility = correct - λ × normalized_cost</code> 가 가장 큰 action 선택
3. Feature 추출	문제 길이, 숫자 개수, 초기 reasoning 길이, 초기 token 수 등 사용
4. Controller 학습	Random Forest가 Oracle action을 예측하도록 학습
5. Calibration	validation set에서 VERIFY / SC-3 선택 threshold 조정

- Question-level feature: question length, word count, numeric token count, arithmetic symbol count
 - Initial-state feature: initial reasoning length, answer format validity, numeric answer extracted
 - Token feature: initial input / output / total tokens
-

Action distribution



- Oracle은 대부분의 문제에서 STOP을 선택하고, 일부 문제에서만 VERIFY를 사용
- RF Controller도 비슷하게 STOP 중심의 보수적인 정책을 학습
- 두 라우터 모두 SC-3는 거의 선택하지 않음
- 추가 compute를 많이 쓰는 것이 핵심이 아니라, **쓸 가치가 있는 문제를 고르는 것이** 핵심.

Model comparison

모델	STOP	VERIFY	Oracle	Best Controller
Qwen2.5-1.5B	0.290	0.440	0.530	0.470
Qwen2.5-3B	0.320	0.540	0.600	0.540

- 3B 모델은 STOP, VERIFY, Oracle, Best Controller 모두에서 1.5B보다 높은 정확도를 보임
 - 특히 VERIFY 성능 향상이 가장 크게 나타남
 - 더 큰 모델은 verification prompt를 활용해 초기 오류를 수정하는 능력이 더 강한 것으로 해석
-

Summary

- Extra compute는 유용하지만, 모든 문제에 동일하게 유용하지는 않았음
 - 이 실험에서는 SC-3보다 VERIFY가 더 효과적인 추가 compute action
 - Oracle과 learned controller 모두 SC-3를 거의 선택하지 않았음
 - 간단한 state feature만으로도 Oracle의 보수적인 라우팅 경향을 일부 학습할 수 있음을 확인
 - 핵심 과제는 **“언제 추가 compute가 오답을 정답으로 바꿀 수 있는가”**를 더 정확히 예측하는 것
-

Reference

- [1] Snell, C., Lee, J., Xu, K., & Kumar, A. "Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters." arXiv:2408.03314, 2024.
 - [2] Wang, X., et al. "Self-Consistency Improves Chain of Thought Reasoning in Language Models." arXiv:2203.11171, 2022.
 - [3] Cobbe, K., et al. "Training Verifiers to Solve Math Word Problems." arXiv:2110.14168, 2021.
 - [4] Zhai, Z., Li, B., Xiao, B., Li, M., & Wang, X. "Adaptive Test-Time Compute Allocation for Reasoning LLMs via Constrained Policy Optimization." arXiv:2604.14853, 2026.
-